ASSIGNMENT 5: ANALYTICAL REPORT

The Role of DevOps and CI/CD in Automating Research Projects

Automating Reproducible Taxonomy Evaluation

Nursultan Serikov, CSE-2501M

Astana IT University, 3 November 2026

1. Research context and the purpose of DevOps

The dissertation topic is the development of taxonomic models of subject areas for a collection of documents. Taxonomy Builder constructs and exports hierarchies; the independent Taxonomy Evaluator compares directed parent-to-child relations with a reference and checks graph structure. This assignment improves the execution and delivery of the evaluator developed in Assignment 3. It does not introduce a new taxonomy-construction method or claim a new experimental result.

DevOps connects development and operation through shared responsibility, short feedback loops, automation, and observable results. In research, operating software includes preparing an environment, running an experiment, preserving its outputs, and enabling another researcher to repeat it. Even a solo researcher benefits from separating implementation, verification, and interpretation as explicit activities. Versioned procedures reduce dependence on remembered terminal commands.

Assignment 1 identified environment and dependency drift as risk R8 and included an explicit reproducibility package. A changed interpreter, input export, or threshold can change a result without an obvious application failure. The operational objective is therefore a traceable chain from code and data to parameters, execution environment, and measured output. Docker and continuous integration make parts of this chain executable and inspectable.

Automation also changes the cost of collaboration. A reviewer can examine a workflow and repeat its checks before relying on a research utility. A failed regression produces immediate evidence instead of becoming a disputed result later. However, a green pipeline establishes only that specified checks passed. It cannot establish that a reference taxonomy is complete, that annotations are independent, or that a sample represents a whole subject area. These remain methodological responsibilities.

2. Continuous integration, delivery, and deployment

Continuous integration evaluates integrated changes frequently against automated checks. GitHub Actions expresses these checks as a versioned YAML workflow with event triggers, jobs, and steps [1]. Here, pushes on every branch and pull requests trigger tests on Python 3.11, 3.13, and 3.14. The matrix preserves the existing compatibility checks; it is not a measurement of every supported operating system.

The practical workflow makes test completion a prerequisite for Docker delivery. It detects the test suite, rejects zero discovered tests, runs the five scientific regression methods, and preserves their logs. Those methods cover known edge counts, export formats, normalization, graph direction and cycles, invalid input, and command-line failure behavior. Python's standard-library unittest supplies the discovery and runner without introducing a runtime dependency [2].

Continuous delivery produces an artifact that has passed the stated checks and is ready to be used. After the test matrix, this workflow builds a Docker image tagged with the checked-out commit SHA, executes the benchmark in that image, and uploads the image archive, source ZIP, JSON result, and diagnostic evidence. Artifact storage preserves outputs beyond the lifetime of a hosted runner [3]. The configured 30-day retention makes a separately downloaded submission archive necessary for longer-term assessment.

Continuous deployment would additionally install a validated version into a specified operating environment automatically. Loading and testing an image within its build job is verification, not deployment to an independent test server. This submission implements CI and artifact delivery. It does not claim the optional server or Kubernetes deployment bonus.

GitHub push events observe changes published to the remote repository. Local commits alone do not trigger them, and several commits in one push do not imply a separate run for every intermediate revision. Recording the event, branch, run number, and actual checkout SHA makes this boundary explicit. Branch-specific evidence is not evidence that the default branch has been updated. Workflow dependencies and failure exit codes matter more than the appearance of a green badge: they determine whether an invalid revision can reach the delivered package.

3. Docker as an execution contract

Docker packages the evaluator with a specific Python runtime and operating-system userspace. The Dockerfile pins Python 3.13.18 on Debian Bookworm slim and the verified image-index digest. Docker documents that tags can move; a digest fixes the selected image content [4]. Deliberate updates are still needed for security maintenance. Pinning the environment is not a promise that it should remain unchanged forever.

The runtime image contains the evaluator and its MIT license. A restrictive build context excludes repository history, reports, credentials, caches, and example data. Predictions and reference files are mounted read-only, while the output directory is mounted separately with the invoking user's permissions. Verification disables networking and uses a read-only container root filesystem. This reflects the existing offline CLI rather than adding an API, database, or service process [5].

For a scientific example, the workflow evaluates the small synthetic energy taxonomy. Without filtering, TP=5, FP=1, and FN=1, so precision, recall, and F1 equal 5/6. At threshold 0.5, the incorrect score-0.40 relation disappears: TP=5, FP=0, FN=1, precision=1, recall=5/6, and F1=10/11. A minimum F1 of 0.9 passes only the filtered example. These hand-checkable values test software behavior; they are not dissertation-model accuracy.

The container check asserts all six numerical values, checks input hashes, writes a report through the output mount, and compares two reports byte for byte in the same image. It also checks a failing quality gate and invalid input. The application preserves exit codes 0, 1, and 2, allowing automation to distinguish a pass, a scientific gate failure, and an input or I/O error. Host and container reports may record different Python versions, so cross-environment byte equality is not required.

The actual Docker execution uses a GitHub-hosted Linux amd64 runner. Local Python tests run on macOS arm64; the installed Docker Desktop could not launch. A multi-platform base image does not prove successful application execution on every architecture. Similarly, the digest fixes userspace, not the host kernel, hardware, or all future build-tool behavior. Evidence should record these limits rather than present containerization as universal reproducibility.

4. Research-scale orchestration and evidence

For a larger multilingual study, Kubernetes Jobs could run independent evaluations for corpus partitions, languages, or parameter settings. A Job tracks tasks that terminate and can retry failed Pods; multiple Pods can execute in parallel [6]. Each task would need an immutable image reference, fixed input identities, an explicit parameter set, a deadline, and persistent outputs. Retries would also require care to avoid duplicate or overwritten experiment records.

The current evaluator completes a tiny batch calculation without network services. A cluster would add administration without resolving its present scientific uncertainties. Assignment 4's multilingual corpus, annotation, and pilot targets remain future research plans. This assignment neither generates those datasets nor verifies those future outcomes. The proportional choice is a hosted CI runner and a downloadable artifact, while describing when orchestration would become useful.

Useful research evidence connects claims to observable records. This implementation records the checkout SHA, trigger, runner and Docker versions, image ID, build log, input hashes, and benchmark output. Screenshots show the actual Actions run; machine-readable logs provide the detail that screenshots cannot. The source ZIP and image archive have checksums. Keeping the reports outside the runtime image avoids coupling a research calculation to document-generation software.

5. Conclusion

DevOps addresses repeated setup work, unnoticed regressions, ambiguous software versions, and lost execution evidence. Its contribution here is a repeatable delivery path for an existing research tool. Scientific validity still requires an independent gold standard, justified normalization conventions, held-out evaluation, and error analysis. Reliable automation makes those methodological decisions easier to inspect; it does not replace them.

References. Official documentation, accessed 3 November 2026.

[1] GitHub. Workflows; Events that trigger workflows.

[2] Python. unittest: Unit testing framework.

[3] GitHub. Store and share data with workflow artifacts.

[4] Docker. Building best practices.

[5] Docker. docker container run.

[6] Kubernetes. Jobs.

Project: taxonomy-evaluator, assignment5-github-actions. Previous Assignments 1–4 provide research context; measured AS5 evidence accompanies the practical report.